Niagara REST API & XML Integration — User Guide

Module: niagaraRestApi v1.0.1 (Niagara 4.14) · **Station:** a Niagara 4.14 test station · **Vendor:** Shanghai Gline Net Co.

Table of Contents

1. Overview & Architecture
2. Authentication (SCRAM-SHA-256)
3. REST API Endpoints
   3.1 GET /api/status
   3.2 GET /api/modules
   3.3 GET /api/types
   3.4 GET /api/config
   3.5 POST /api/batch (10 Operations)
4. Verified Test Samples
   4.1 Modbus TCP Device
   4.2 MQTT + BooleanWritable
   4.3 Add + Link + Rename
   4.4 Deep Copy + Rename
   4.5 Add + Link + Unlink + Remove
5. XML (oBIX) Usage Guide
6. Python Client Tools
7. Building & Deploying
8. Known Limitations
9. Alarms & History — Extensions via REST
10. Reading Alarms & History Data — Web Box

1. Overview & Architecture

The **niagaraRestApi** module is a custom Niagara N4 module that embeds an HTTP server inside a Niagara Station. It exposes a JSON-based REST API for programmatic interaction with the station.

Architecture

```
RestApiService  (extends BComponent)
 ├─ enabled  (boolean)
 ├─ port     (int, default 8082)
 ├─ started() → Creates HttpServer
 └─ stopped() → Stops  HttpServer

Handlers:
  Status  → GET  /api/status
  Modules → GET  /api/modules
  Types   → GET  /api/types?module=xxx
  Config  → GET  /api/config?path=/xxx
  Batch   → POST /api/batch
```

- JDK built-in HttpServer — no external container.
- JSON-only request/response.
- **Every request must be authenticated** (HTTP Basic Auth) — see Section 2.
- Batch mutation model: all writes via `POST /api/batch`.

2. Authentication

2.1 REST API — Basic Auth on every request

The REST API is **authenticated by default**. Every request must carry HTTP Basic credentials; a missing or wrong credential is rejected with **HTTP 401**.

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[{"OP":"list","PATH":"/"}]'
```

The **only** endpoint exempt from authentication is `GET /api/status`, so a monitoring system can use it as a liveness probe:

```
# Unauthenticated liveness probe — the sole auth-exempt endpoint
curl http://<HOST>:8082/api/status
{"status":"ok","name":"glineRestApi","version":"2.4","port":8082}
```

Credentials are configurable at runtime on the service component and persist across station restarts. In addition, a **CIDR allow-list** can restrict access to trusted network segments; a request from outside the list is rejected with **HTTP 403** (loopback is always allowed as a recovery path).

Request	Result	Note
No credentials	**401**	Rejected
Wrong credentials	**401**	Rejected
Valid credentials, outside allow-list	**403**	Rejected
Valid credentials, inside allow-list	200	Accepted
`GET /api/status` without credentials	200	Liveness probe — the only exemption

■ **Even though the API is authenticated, still do not expose it directly to the public internet — put it behind a firewall or reverse proxy.**

2.2 Built-in XML API — SCRAM-SHA-256

Niagara N4+ uses SCRAM-SHA-256 (5-step handshake). Legacy form login does **NOT** work.

SCRAM Handshake (Python)

```
import hashlib, hmac, base64, os, requests

HOST = "http://<HOST>:80"
USER = "<USER>"
PASS = "<PASS>"

s = requests.Session()
n = base64.b64encode(os.urandom(16)).decode()[:16]
c1 = "n,,n=%s,r=%s" % (USER, n)
hdrs = {"Content-Type": "application/x-niagara-login-support",
        "X-Requested-With": "XMLHttpRequest"}
r1 = s.post(HOST+"/j_security_check",
    data="action=sendClientFirstMessage&clientFirstMessage="+c1,
    headers=hdrs, timeout=10)

d = {}
for i in r1.text.strip().split(","):
    if "=" in i:
        k, v = i.split("=", 1); d[k.strip()] = v.strip()

salt  = base64.b64decode(d["s"]); iters = int(d["i"])
sp    = hashlib.pbkdf2_hmac("sha256", PASS.encode(), salt, iters, 32)
ck    = hashlib.sha256(hmac.new(sp, b"Client Key", hashlib.sha256).digest()).digest()
sk    = hashlib.sha256(ck).digest()
auth_msg = ("n=%s,r=%s,r=%s,s=%s,i=%s,c=biws,r=%s" % (USER,n,d["r"],d["s"],d["i"],d["r"])).encode()
cs    = hmac.new(sk, auth_msg, hashlib.sha256).digest()
pr    = bytes([a ^ b for a, b in zip(ck, cs)])
cf    = "c=biws,r=%s,p=%s" % (d["r"], base64.b64encode(pr).decode())

s.post(HOST+"/j_security_check",
    data="action=sendClientFinalMessage&clientFinalMessage="+cf, headers=hdrs, timeout=10)
s.post(HOST+"/j_security_check", data={"j_username": USER, "j_password": ""}, timeout=10)

r = s.get(HOST+"/obix/config/", timeout=10)
print(r.text[:500])
```

3. REST API Endpoints Reference

3.1 GET /api/status

Check if the REST API service is running.

```
curl -u "<USER>:<PASS>" http://<HOST>:8082/api/status
{"status":"ok","name":"glineRestApi","version":"2.4","port":8082}
```

The port is configurable on the service component; `8082` is the default. Replace `<HOST>:8082` with your station's actual host and port.

3.2 GET /api/modules

Informational stub. This endpoint does not enumerate modules — it returns a fixed hint pointing at `/api/config`.

```
curl -u "<USER>:<PASS>" http://<HOST>:8082/api/modules
{"note":"use /api/config to browse station tree"}
```

3.3 GET /api/types

Informational stub. Returns a fixed hint listing common types; it does not enumerate the type dictionary for a module. Browse the station tree via `/api/config`, or run a BQL query (Section 3.5), to enumerate types.

```
curl -u "<USER>:<PASS>" "http://<HOST>:8082/api/types?module=control"
{"note":"use /api/config to browse types; common types: baja:Folder, ..."}
```

Module	Common Types	Description
baja	Folder	Container
control	NumericWritable, BooleanWritable	Point
modbusTcp	ModbusTcpNetwork, ModbusTcpDevice	TCP net/dev
modbusCore	ModbusClientNumericProxyExt	Proxy ext
kitControl	Add, Ramp, SineWave	Simulation

3.4 GET /api/config

Browse the config tree at any path.

```
curl -u "<USER>:<PASS>" "http://<HOST>:8082/api/config?path=/"
{"name":"Station","type":"Station","children":[
  {"name":"AlarmService","type":"AlarmService"},
  {"name":"Drivers","type":"Driver"}]}
```

3.5 POST /api/batch — Core Operation

All mutations go through this endpoint. Send a JSON array of operation objects.

Common Fields

Field	Type	Description	Applies To
OP	string	Operation type	ALL
PATH	string	Component path	ALL
T	string	Type module:Type	add
N	string	Name	add, mod, link
NEW	string	New name	rename, cp
V	any	Value	mod
TARGET	string	Target path	link, unlink
TARGET_N	string	Target slot	link
VAL	any	Invoke value	invoke
VAL_TYPE	string	Type hint	invoke
TARGET_PATH	string	Dest path	cp

10 Operations

Op	Action	Example
1. ADD	Create component	`{"OP":"add","PATH":"/","T":"baja:Folder","N":"myFolder"}`
2. LINK	Connect	`{"OP":"link","PATH":"/src","N":"out","TARGET":"/tgt","TARGET_N":"in10"}`
3. RENAME	Rename (preserves links)	`{"OP":"rename","PATH":"/folder/old","NEW":"new"}`
4. CP	Deep copy + rebuild links	`{"OP":"cp","PATH":"/folder","NEW":"copy","TARGET_PATH":"/"}`
5. MOD	Write property	`{"OP":"mod","PATH":"/device","N":"ipAddress","V":"<IP>"}`
6. UNLINK	Remove link	`{"OP":"unlink","TARGET":"/tgtPoint"}`
7. REMOVE	Delete component	`{"OP":"remove","PATH":"/parent/child"}`
8. READ	Read property	`{"OP":"read","PATH":"/device","N":"ipAddress"}`
9. LIST	List children	`{"OP":"list","PATH":"/myFolder"}`
10. INVOKE	Call action	`{"OP":"invoke","PATH":"/point/set","VAL":"42","VAL_TYPE":"num"}`

MOD Limitations

- Enum types (regType, dataType) cannot be written via MOD.
- BFacets (deviceFacets) cannot be written via MOD.

4. Verified Test Samples

All tested on a Niagara 4.14 station with default admin credentials.

4.1 Create Modbus TCP Device

Creates a Modbus TCP device with 3 points (OnOff, Mode, SetTemp) at address 2, with a device IP and ping verification.

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/Drivers/ModbusTcpNetwork","T":"modbusTcp:ModbusTcpDevice","N":"T8600_new"},
{"OP":"mod","PATH":"/.../T8600_new","N":"deviceAddress","V":2},
{"OP":"mod","PATH":"/.../T8600_new","N":"ipAddress","V":"<IP>"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"OnOff"},
{"OP":"add","PATH":"/.../OnOff","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../OnOff/proxyExt","N":"dataAddress","V":"2|decimal"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"Mode"},
{"OP":"add","PATH":"/.../Mode","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../Mode/proxyExt","N":"dataAddress","V":"3|decimal"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"SetTemp"},
{"OP":"add","PATH":"/.../SetTemp","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../SetTemp/proxyExt","N":"dataAddress","V":"4|decimal"},
{"OP":"invoke","PATH":"/Drivers/ModbusTcpNetwork/T8600_new/ping"}]'
```

4.2 MQTT + BooleanWritable

Creates 10 BooleanWritable points and links them to the MqttClient output.

```
[{"OP":"add","PATH":"/MQTT","T":"control:BooleanWritable","N":"Point1"}, ... ,
 {"OP":"add","PATH":"/MQTT","T":"control:BooleanWritable","N":"Point10"}]
```

4.3 Add + Link + Rename Flow

Create, link, rename — verify the link survives rename.

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"testFolder"},
 {"OP":"add","PATH":"/testFolder","T":"control:NumericWritable","N":"src"}, ...]
```

4.4 Deep Copy (CP)

Create a folder with links, deep copy — internal links are rebuilt.

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"original"},
 {"OP":"add","PATH":"/original","T":"control:NumericWritable","N":"point1"}, ...]
```

4.5 Add + Link + Unlink + Remove

Full lifecycle: add, link, unlink, remove.

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"lifecycle"},
 {"OP":"add","PATH":"/lifecycle","T":"control:NumericWritable","N":"src"}, ...]
```

5. XML (oBIX) Usage Guide

The built-in XML API is an XML/JSON protocol for building automation. Niagara N4 exposes it under `/obix/`.

Protocol Basics

- XML by default, JSON with an `Accept` header.
- Components as `<obj is="module:Type">`.
- Child refs: `<ref href="..." name="..." />` — follow href to read.
- Include `X-Requested-With: XMLHttpRequest` header.
- Requires SCRAM-SHA-256 auth (Section 2).

Paths

Path	Method	Description
`/obix/`	GET	Server info
`/obix/about/`	GET	Details
`/obix/config/`	GET	Config root
`/obix/config/Services/`	GET	Services
`/obix/config/Drivers/`	GET	Drivers
`/.../~alarmQuery/`	POST	Alarm query
`/.../~historyQuery/`	GET	History
`/ord?station:|slot:/`	GET	HTML view

Alarm Query

Authenticate with the SCRAM handshake (Section 2.2) to obtain a session, then:

```
curl --user "<USER>:<PASS>" --negotiate -X POST \
  "http://<HOST>/obix/config/Services/AlarmService/~alarmQuery/" \
  -H "Content-Type: text/xml" --data-binary @query.xml
```

History Query

```
curl --user "<USER>:<PASS>" \
  "http://<HOST>/obix/config/Services/HistoryService/historyGroupings/"
curl --user "<USER>:<PASS>" \
  "http://<HOST>/obix/config/.../~historyQuery/?start=-7days"
```

XML API vs REST API

Feature	XML API	REST API
Auth	SCRAM required	None
Format	XML/JSON	JSON

Read	Follow	GET /api/config
Create	Not supported	POST /api/batch (ADD)
Write	PUT with value	POST /api/batch (MOD)
Alarm query	POST /~alarmQuery/	Not built-in
History	GET /~historyQuery/	Not built-in
Batch ops	Not supported	10 ops

6. Python Client Tools

6.1 niagara-rest-client.py

Universal REST client: login, status, browse, BQL, history, overrides, alarms.

```
python niagara-rest-client.py <host:port> --user <USER> --password <PASS> \
  [--status | --config <p> | --bql <q> | --batch <f> | --history ...]
```

6.2 niagara-bql-query.py

BQL CLI tool with CSV export.

```
python niagara-bql-query.py --host <HOST> --user <USER> --password <PASS> \
  --query "SELECT * FROM control:BooleanWritable"
```

6.3 alarm_to_csv.py

Alarm export: ProgramService BSH or Python external client.

```
python alarm_to_csv.py --host <HOST> --user <USER> --password <PASS> \
  --output alarms.csv [--days N | --all]
```

6.4 Exploration Scripts

- **explore_station.py:** REST-based station exploration.
- **explore_station_v2.py:** Full SCRAM + XML parsing. Extracts refs, searches Modbus/BACnet.
- **test_connection.py:** Quick connectivity test.

7. Building & Deploying

Prerequisites

- Niagara 4.14 SDK
- Java JDK 8 (Zulu 8)
- Gradle wrapper

Build

```
cd <project-dir>
gradle wrapper --gradle-version 8.2
gradle clean build -x test
# Output: niagaraRestApi.jar + niagaraRestApi-rt.jar
```

Deploy

```
copy niagaraRestApi-rt.jar <station-home>/rt/
# OR: <NiagaraHome>/modules/
# Restart station. Services → New → RestApiService
# Configure port & enabled, restart again.
```

Dependencies

baja 4.14 · nre 4.14 · control 4.14 · modbusCore 4.14 · modbusTcp 4.14

8. Known Limitations

Limitation	Detail
Authentication	**Required on every request** (HTTP Basic Auth); the only exemption is `GET /api/status` as a liveness probe.
Enum Properties	regType, dataType: cannot write via MOD (BString→enum fails).
BFacets Properties	deviceFacets: cannot write via MOD.
XML API Cannot Create	Niagara N4 XML API does not support PUT/POST creation.
REST No Alarm Query	No alarm **query** endpoint in REST. Configure extensions via REST (Section 9) and read data via Web Box (Section 10).

Disable Needs Restart	Disabling RestApiService requires a station restart.

9. Alarms & History — Extensions via REST

Alarm and history capture is configured by attaching **extensions** to a point. These are ordinary `BComponent` children, so they can be created and configured through the REST API — no GUI required.

Extension types

Type	Purpose
`alarm:AlarmSourceExt`	Alarm source extension
`history:NumericCovHistoryExt`	Numeric history, change-of-value
`history:BooleanCovHistoryExt`	Boolean history, change-of-value
`history:EnumCovHistoryExt`	Enum history, change-of-value
`history:StringCovHistoryExt`	String history, change-of-value
`history:NumericIntervalHistoryExt`	Numeric history, fixed interval

Attach a history extension and enable it

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/myFolder/pt","T":"history:NumericCovHistoryExt","N":"he"},
{"OP":"mod","PATH":"/myFolder/pt/he","N":"enabled","V":true}
]'
```

After `enabled=true` the extension reports `active: true` — history collection is live.

Attach an alarm extension and configure it

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/myFolder/pt","T":"alarm:AlarmSourceExt","N":"ae"},
{"OP":"mod","PATH":"/myFolder/pt/ae","N":"alarmClass","V":"/AlarmClass"},
{"OP":"mod","PATH":"/myFolder/pt/ae","N":"timeDelay","V":5000}
]'
```

Known MOD limits for extensions

Slot	Behaviour
`enabled`	Boolean — writable; activates a history extension
`alarmClass`	String — writable
`timeDelay`	`RelTime` — write a **raw millisecond number**, not `"5s"`
`historyName` / `toOffnormalText`	`BFormat` — **cannot** be written via MOD
`alarmEnable`	`AlarmTransitionBits` — **cannot** be written via MOD

Extensions created via REST remain fully visible and editable in Workbench.

10. Reading Alarms & History Data — Web Box

The REST API deliberately has no alarm or history endpoints. That data is available through the station's **Web Box** feature, which can be driven programmatically for automated extraction.

Web Box query endpoints

Endpoint	Method	Purpose
`/wsbox/query/data/<ord>`	GET	Read history records for a point (paged, sortable)
`/wsbox/query/source/<ord>`	GET	History point source / metadata
`/wsbox/query/lastTime/<ord>`	GET	Timestamp of the most recent record

Query parameters

- `offset` / `limit` — pagination window.
- `sortColumn=timestamp` and `descending=false` — chronological order.
- `timeRange=startTime=…;endTime=…` — bounded time window.

Slot path encoding

History points are addressed by an **ord** of the form `history:/<station>/<historyName>`. When passing an ord in a URL it must be escaped — the mapping includes `:` → `$3a`, `/` → `$2f`, `?` → `$3f`, `=` → `$3d`, `&` → `$26`, `$` → `$24`.

■ **Escaping `&` is mandatory.** A raw `&` is parsed as an HTTP parameter separator and the query silently returns an empty result set.

Automation notes

- Requests must be sent as **GET**.
- The response is a chunked **NDJSON** stream — read it with a streaming client (e.g. browser `XMLHttpRequest` with `onprogress`), not a single blocking read.
- The CSRF token must be obtained inside the station page context (`csrfUtil.getCsrfToken()`) and sent in the `x-niagara-csrfToken` header; the station session must already be authenticated.

With these pieces in place, alarm and history extraction can be fully automated — no fixed REST endpoint is required for it.

For commercial customization, integration, or deployment support, please contact:

Shanghai Gline Net Co.
Email: jason.zhang@gline-net.com
Tel / WhatsApp: +86 138 0199 0968

© 2026 Shanghai Gline Net Co. — End of Document